Linux

Filesysteem structuur

Absoluut Pad

Een absoluut pad geeft de volledige locatie van een bestand of directory aan, te beginnen bij de root-directory (`/`). Het absolute pad specificeert de locatie van het bestand of de map zonder afhankelijk te zijn van de huidige werkdirectory.

Voorbeeld:

```
/home/username/documenten/project.txt
```

Dit pad begint bij de root (`/`) en leidt naar een bestand genaamd `project.txt` in de map `documenten` binnen de map `username` .

Relatief Pad

Een relatief pad geeft de locatie van een bestand of directory aan ten opzichte van de huidige werkdirectory. Het gebruikt `.` (de huidige map) of `..` (de bovenliggende map) om de relatie aan te geven.

Voorbeeld:

```
./project.txt
```

Dit pad verwijst naar een bestand genaamd `project.txt` in de huidige directory.

```
../project.txt
```

Dit pad verwijst naar een bestand genaamd `project.txt` in de bovenliggende directory van de huidige map.

Filesystem Commando's

Filesystem commando's zijn Linux-opdrachten die worden gebruikt om te navigeren, beheren en organiseren binnen het bestandssysteem. Ze stellen gebruikers in staat om mappen en bestanden te creëren, te verwijderen, te verplaatsen, te kopiëren, en de structuur van het bestandssysteem te bekijken. Deze commando's zijn essentieel voor het beheren van de inhoud en de structuur van een Linux-systeem.

Print Working Directory [`pwd`]

- **Beschrijving**: Het `pwd` -commando toont het pad van de huidige werkdirectory. Dit is nuttig om te weten waar je je bevindt in de mappenstructuur.
- **Voorbeeld**:

  ```
  pwd
  ```

  Dit commando toont iets als `/home/gebruiker/documenten` , wat het huidige pad is.

Change Directory [`cd`]

- **Beschrijving**: Het `cd` -commando verandert de huidige werkdirectory naar een opgegeven map. Je kunt ook snel naar de home-directory of de vorige directory navigeren.
- **Belangrijkste Parameters**:
  - `cd ~` : Ga naar de home-directory.
  - `cd ..` : Ga naar de bovenliggende directory.
  - `cd -` : Ga naar de vorige directory.
- **Voorbeeld**:

  ```
  cd /var/log
  ```

  Dit commando verandert de huidige map naar `/var/log` .

List Directory Contents [`ls`]

- **Beschrijving**: Het `ls` -commando toont de inhoud van een directory, zoals bestanden en submappen. Het kan ook details weergeven zoals permissies, eigendom en bestandsgroottes.
- **Belangrijkste Parameters**:
  - `-l` : Toont een gedetailleerde lijst met bestandsinformatie (lange lijst).
  - `-a` : Toont ook verborgen bestanden (die beginnen met een `.`).
  - `-h` : Toont de bestandsgroottes in een leesbare vorm (bijv. KB, MB).
- **Voorbeeld**:

```
ls -lah
```

Dit commando toont een gedetailleerde lijst van alle bestanden, inclusief verborgen bestanden, in een leesbaar formaat.

Mapstructuur weergeven in een boomvorm [`tree`]

- **Beschrijving**: Het `tree` -commando toont de mappenstructuur in een boomvorm, zodat je een overzicht hebt van de inhoud van een directory en de subdirectories.
- **Belangrijkste Parameters**:
  - `-L [niveau]` : Toont de boomstructuur tot een bepaald niveau van diepte.
  - `-a` : Toont ook verborgen bestanden en mappen.
- **Voorbeeld**:

```
tree -L 2
```

Dit commando toont de mappenstructuur tot een diepte van 2 niveaus.

Maak een leeg bestand aan [`touch`]

- **Beschrijving**: Het `touch` -commando wordt gebruikt om een nieuw, leeg bestand aan te maken of de toegangstijd van een bestaand bestand bij te werken.
- **Voorbeeld**:

```
touch nieuw_bestand.txt
```

Dit commando maakt een nieuw leeg bestand genaamd `nieuw_bestand.txt` .

Maak een nieuwe map aan [`mkdir`]

- **Beschrijving**: Het `mkdir` -commando maakt een nieuwe directory aan. Je kunt ook meerdere niveaus van directories in één keer maken.
- **Belangrijkste Parameters**:
  - `-p` : Maak parent-directories aan indien nodig.
- **Voorbeeld**:

```
mkdir nieuwe_map
mkdir -p hoofdmap/submap
```

Het eerste commando maakt een directory genaamd `nieuwe_map` aan. Het tweede commando maakt een subdirectory `submap` binnen `hoofdmap` .

Verwijder bestanden en mappen [`rm`]

- **Beschrijving**: Het `rm` -commando wordt gebruikt om bestanden of mappen te verwijderen. Wees voorzichtig, want verwijderen is meestal onomkeerbaar.
- **Belangrijkste Parameters**:
  - `-r` : Recursief verwijderen (verwijdert een directory en alle inhoud).
  - `-f` : Forceer het verwijderen zonder bevestiging.
- **Voorbeeld**:

  ```
  rm bestand.txt
  rm -rf mapnaam
  ```

  Het eerste commando verwijdert `bestand.txt` . Het tweede commando verwijdert de directory `mapnaam` inclusief alle inhoud.

Verplaats of hernoem bestanden en mappen [`mv`]

- **Beschrijving**: Het `mv` -commando wordt gebruikt om bestanden of mappen te verplaatsen of te hernoemen. Je kunt bestanden verplaatsen naar een andere directory of de naam wijzigen.
- **Voorbeeld**:

  ```
  mv bestand.txt /home/gebruiker/map/
  mv oudenaam.txt nieuwenaam.txt
  ```

  Het eerste commando verplaatst `bestand.txt` naar `/home/gebruiker/map/` . Het tweede commando wijzigt de naam van `oudenaam.txt` naar `nieuwenaam.txt` .

Kopieer bestanden en mappen [`cp`]

- **Beschrijving**: Het `cp` -commando wordt gebruikt om bestanden en mappen te kopiëren. Je kunt bestanden kopiëren naar een andere directory of de kopie een andere naam geven.
- **Belangrijkste Parameters**:
  - `-r` : Recursief kopiëren (kopieert een map en alle inhoud).
  - `-i` : Vraagt om bevestiging voordat bestaande bestanden worden overschreven.
- **Voorbeeld**:

  ```
  cp bestand.txt kopie_bestand.txt
  cp -r mapnaam kopie_map
  ```

  Het eerste commando kopieert `bestand.txt` naar `kopie_bestand.txt` . Het tweede commando kopieert de directory `mapnaam` inclusief alle inhoud naar `kopie_map` .

Tekst Commando's

Tekstcommando's zijn Linux-opdrachten die specifiek worden gebruikt om tekstbestanden te bekijken, zoeken, manipuleren, of analyseren. Ze helpen bij het uitvoeren van eenvoudige of complexe bewerkingen op tekst, zoals het bekijken van inhoud, het zoeken naar specifieke woorden, het sorteren van regels, en het filteren van gegevens.

Print bestanden [`cat`]

- **Beschrijving**: Het `cat` -commando leest een bestand en toont de inhoud ervan. Je kunt ook meerdere bestanden samenvoegen of de uitvoer omleiden naar een ander bestand.
- **Belangrijkste Parameters**:
  - `-n` : Nummer de regels in de uitvoer.
  - `-b` : Nummer alleen de niet-lege regels.
- **Voorbeeld**:

```
# file.txt
Line 1
Line 2

Line 4
Line 5


cat -n file.txt
# 1  Line 1
# 2  Line 2
# 3
# 4  Line 4
# 5  Line 5

cat -b file.txt
# 1  Line 1
# 2  Line 2

# 3  Line 4
# 4  Line 5
```

Dit commando toont de inhoud van `file.txt` op het scherm,al dan niet met lijnregels.

Toon het begin van een bestand [`head`]

- **Beschrijving**: Het `head` -commando toont standaard de eerste 10 regels van een bestand. Je kunt zelf het aantal regels specificeren met een parameter.
- **Belangrijkste Parameters**:
  - `-n [aantal]` : Geeft het aantal weer te geven regels op.
- **Voorbeeld**:

```
# file.txt
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
```

```
head -n 3 file.txt
# Line 1
# Line 2
# Line 3
```

Dit commando toont de eerste 3 regels van `file.txt` . Als het aantal lijnregels niet wordt meegegeven worden er standaard 10 regels getoond. Let op, lege regels tellen ook mee als regels.

Toon het einde van een bestand [`tail`]

- **Beschrijving**: Het `tail` -commando toont standaard de laatste 10 regels van een bestand. Het wordt vaak gebruikt om de laatste wijzigingen in een logbestand te bekijken.
- **Belangrijkste Parameters**:
  - `-n [aantal]` : Geeft het aantal weer te geven regels op.
  - `-f` : Volgt het bestand in realtime, toont nieuwe regels zodra ze worden toegevoegd.
- **Voorbeeld**:

```
# file.txt
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
```

```
tail -n 2 file.txt
# Line 5
# Line 6
```

Dit commando toont de laatste 2 regels van `file.txt` .

Zoek naar een patroon in bestanden [`grep`]

- **Beschrijving**: Het `grep` -commando zoekt naar een patroon in bestanden en toont de regels die overeenkomen met dat patroon.
- **Belangrijkste Parameters**:
  - `-i` : Zoek ongevoelig voor hoofdletters.
  - `-r` : Zoek recursief in submappen.
  - `-v` : Toon de regels die **niet** overeenkomen met het patroon.
- **Voorbeeld**:

```
# example.txt
Hello World
This is a test file.
hello again
Testing grep command
Another line here
test 123
```

```
grep "test" example.txt
# This is a test file.
# test 123

grep -i "hello" example.txt
# Hello World
# hello again

grep -r "test" /path/to/directory
# /path/to/directory/example.txt:This is a test file.
# /path/to/directory/example.txt:test 123

grep -v "test" example.txt
# Hello World
# hello again
# Testing grep command
# Another line here
```

Let op, met het commando `grep "test" example.txt` wordt regel 4 niet getoond omwille van de hoofdlettergevoeligheid.

Verwijder secties van elke regel van bestanden [`cut`]

- **Beschrijving**: Het `cut` -commando wordt gebruikt om specifieke kolommen of velden uit een bestand te halen. Je kunt een scheidingsteken en veldspecificatie opgeven.
- **Belangrijkste Parameters**:
  - `-d [scheidingsteken]` : Het scheidingsteken dat kolommen scheidt (bijv. een komma).
  - `-f [velden]` : De velden die je wilt extraheren.
- **Voorbeeld**:

```
# bestand.csv
ID,Name,Age,Occupation
1,John Doe,30,Engineer
2,Jane Smith,25,Designer
3,Emily Davis,28,Doctor
4,Michael Brown,35,Teacher
```

```
cut -d "," -f 2 bestand.csv
# Name
# John Doe
# Jane Smith
# Emily Davis
# Michael Brown
```

Dit commando haalt de tweede kolom uit `bestand.csv` op, waarbij de komma als scheidingsteken wordt gebruikt. Indien je meerdere kolommen wil ophalen plaats je een komma (`,`) voor een selectie van kolommen, of koppelteken (`-`) voor een range van kolommen.

Sorteer regels van tekstbestanden [`sort`]

- **Beschrijving**: Het `sort` -commando sorteert de regels van een tekstbestand op alfabetische of numerieke volgorde.
- **Belangrijkste Parameters**:
  - `-r` : Sorteer in omgekeerde volgorde.
  - `-n` : Sorteer op numerieke waarden.
- **Voorbeeld**:

```
# bestand.txt
apple
banana
apple
apple
orange
banana
```

```
sort bestand.txt
# apple
# apple
# apple
# banana
# banana
# orange
```

Dit commando sorteert de regels in `bestand.txt` alfabetisch.

Print of verwijder dubbele regels [`uniq`]

- **Beschrijving**: Het `uniq` -commando verwijdert dubbele regels uit een bestand, maar het werkt alleen op opeenvolgende duplicaten, dus het wordt vaak gecombineerd met `sort` .
- **Belangrijkste Parameters**:
  - `-c` : Toon het aantal keren dat een regel voorkomt.
- **Voorbeeld**:

```
# bestand.txt
apple
banana
apple
apple
orange
banana
```

```
uniq -c bestand.txt
# 1 apple
# 1 banana
# 2 apple
# 1 orange
# 1 banana

sort bestand.txt | uniq -c
# 3 apple
# 2 banana
# 1 orange
```

Zie hoe we door eerst te sorteren ons beoogd resultaat kunnen bekomen.

Woorden, regels en bytes tellen [`wc`]

- **Beschrijving**: Het `wc` -commando telt regels, woorden en tekens in een bestand.
- **Belangrijkste Parameters**:
  - `-l` : Tel het aantal regels.
  - `-w` : Tel het aantal woorden.
  - `-c` : Tel het aantal tekens (bytes).
- **Voorbeeld**:

```
# file.txt
Hello World
This is an example file.
It contains several lines.


wc -l -w -c file.txt
# 3  11  64 file.txt
```

Dit commando toont het aantal regels, woorden en tekens in `file.txt` . Let op, spaties, en speciale tekens (`/n` , `/t`) zijn ook characters (bytes).

Toon een regel tekst [`echo`]

- **Beschrijving**: Het `echo` -commando drukt een lijn tekst af naar de standaard uitvoer (meestal de terminal).
- **Belangrijkste Parameters**:
  - `-e` : Hiermee worden speciale tekens geïnterpreteerd, zoals nieuwe regels (`\n`).
- **Voorbeeld**:

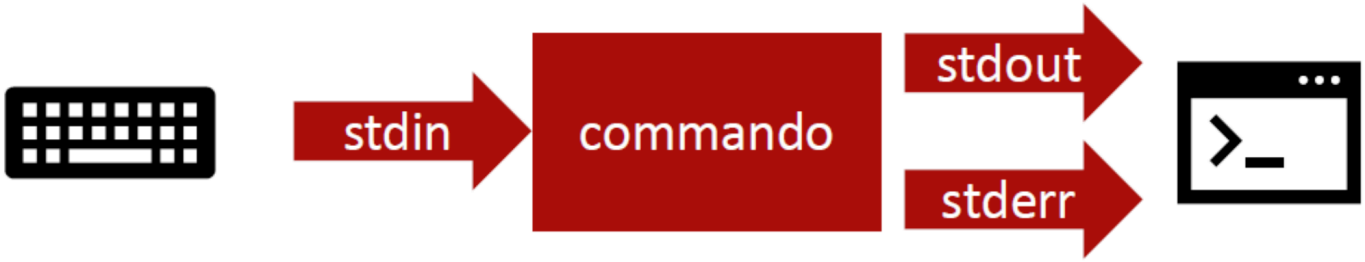
  ```
  echo "Hello, World!"
  # Hello, World!

  echo -e "Hello\nWorld"
  # Hello
  # World
  ```

Streams

Input en Output Streams

In Linux zijn **streams** een fundamenteel concept voor communicatie tussen de gebruiker, het besturingssysteem en programma's. Deze worden voornamelijk gebruikt voor **input** (invoer van data), **output** (uitvoer van data), en **error handling** (foutafhandeling). Laten we de drie hoofdstreams verkennen: **stdin**, **stdout**, en **stderr**.



Standard Input (stdin)

- **Identifier**: `0`
- **Beschrijving**: Dit is de standaard invoerbron voor een programma. Typisch is dit invoer vanaf het toetsenbord, maar stdin kan omgeleid worden naar andere bronnen, zoals bestanden of output van andere programma's.
- **Gebruik**: Invoeren van data naar een programma via de standaard invoerbron.
- **Voorbeelden**:
  - **Manuele invoer**: Wanneer je een commando zoals `cat` uitvoert zonder bestand, wordt standaard invoer verwacht van het toetsenbord.

    ```
    cat
    ```

    Het commando wacht op invoer via het toetsenbord totdat je op `Ctrl + D` drukt om de invoer af te sluiten.
  - **Invoer van een bestand**:

    ```
    cat < invoerbestand.txt
    ```

    Hier wordt de inhoud van `invoerbestand.txt` als invoer (stdin) aan het `cat` -commando gegeven.

Standard Output (stdout)

- **Identifier**: `1`
- **Beschrijving**: Dit is de standaard uitvoerbron van een programma. Standaard wordt de uitvoer naar de terminal gestuurd, maar het kan ook naar een bestand of een ander programma worden omgeleid.
- **Gebruik**: Het tonen van resultaten of informatie op het scherm.
- **Voorbeelden**:
  - **Standaard uitvoer naar de terminal**:

    ```
    echo "Hello, World!"
    ```

Dit commando toont "Hello, World!" in de terminal via stdout.

- **Uitvoer omleiden naar een bestand**:

```
echo "Hello, World!" > output.txt
```

Hier wordt de standaard uitvoer omgeleid naar `output.txt`, wat betekent dat de tekst niet op het scherm verschijnt maar wordt opgeslagen in het bestand.

Standard Error (stderr)

- **Identifier**: `2`
- **Beschrijving**: Deze stream is bedoeld voor het versturen van foutmeldingen. Dit voorkomt dat foutmeldingen en reguliere uitvoer samenkomen, wat anders tot verwarring kan leiden. Net als stdout kan stderr worden omgeleid.
- **Gebruik**: Weergeven van foutmeldingen of waarschuwingen.
- **Voorbeelden**:
  - **Standaard foutmeldingen naar de terminal**:

```
ls niet_bestaand_bestand
```

Dit commando probeert een bestand te zoeken dat niet bestaat, en de foutmelding verschijnt in de terminal.

  - **Fouten omleiden naar een bestand**:

```
ls niet_bestaand_bestand 2> error_log.txt
```

Hier wordt de foutmelding in `error_log.txt` opgeslagen.

Redirection (Omleiding) in Linux

Redirection is een techniek om de standaardstroom van een programma (stdin, stdout of stderr) naar een bestand of een ander programma te sturen. Het maakt automatisering en gegevensverwerking veel efficiënter.

Belangrijke Redirection Symbolen:

- **`>`** : Omleiden van stdout naar een bestand, en overschrijft de bestaande inhoud.

```
echo "Nieuwe tekst" > bestand.txt
```

- **`>>`** : Omleiden van stdout en toevoegen aan een bestand zonder te overschrijven.

```
echo "Extra tekst" >> bestand.txt
```

- **<** : Omleiden van stdin van een bestand.

  ```
  cat < bestand.txt
  ```

- **2>** : Omleiden van stderr naar een bestand.

  ```
  ls niet_bestaand_bestand 2> fouten.log
  ```

- **&>** : Omleiden van zowel stdout als stderr naar hetzelfde bestand.

  ```
  command &> uitvoer_en_fouten.txt
  ```

Wanneer het bestand waarnaar we de **stdout** of **stderr** omleiden niet bestaat zal dit door Linux aangemaakt worden. Tenzij je niet de juiste rechten hebt om bestanden te creëren of aan te passen binnen de huidige directory.

Wanneer het bestand dat we gebruiken als invoer (**stdin**) niet bestaat zal je wel een error krijgen.

```
cat < non_existing_file.txt
# cat: non_existing_file.txt: No such file or directory
```

Pipes (|) en Filtercommando's

Pipes worden gebruikt om de uitvoer van één commando als invoer te sturen naar een ander commando. Dit is een krachtige techniek in Linux om complexe bewerkingen uit te voeren.

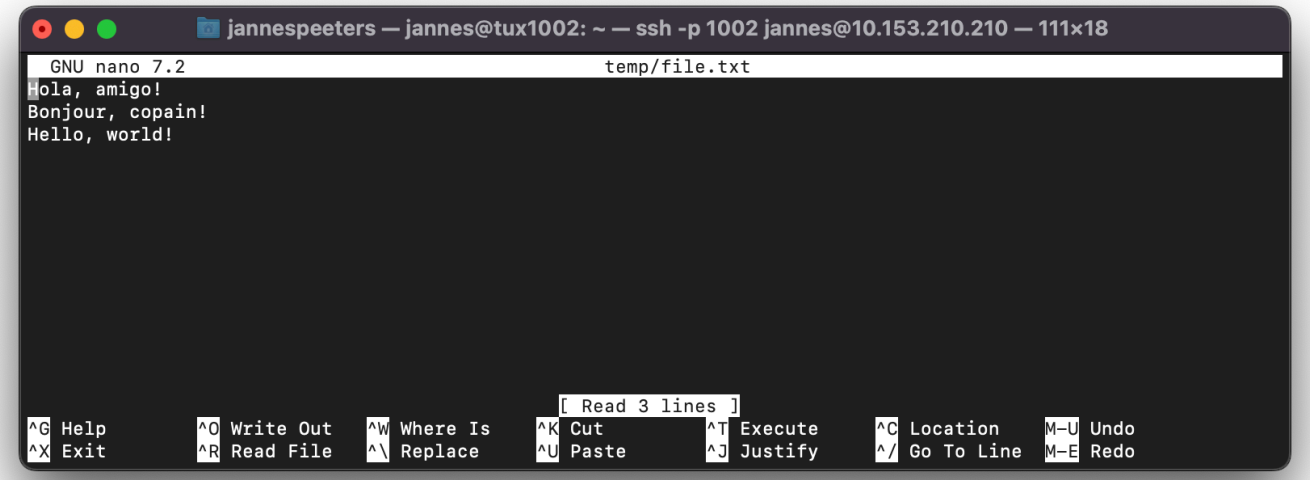
Voorbeeld van een Pipe:

```
ls -l | grep ".txt"
```

In dit voorbeeld stuurt het `ls -l` -commando een gedetailleerde lijst van alle bestanden naar `grep` , die alleen de bestanden met de extensie `.txt` toont.

Tekstbestanden Bewerken

De `nano` Editor



`nano` is een eenvoudige en gebruiksvriendelijke teksteditor. Je kunt direct beginnen met typen om tekst toe te voegen of te bewerken.

Installate

```
sudo apt-get install nano
```

Basis Commando's:

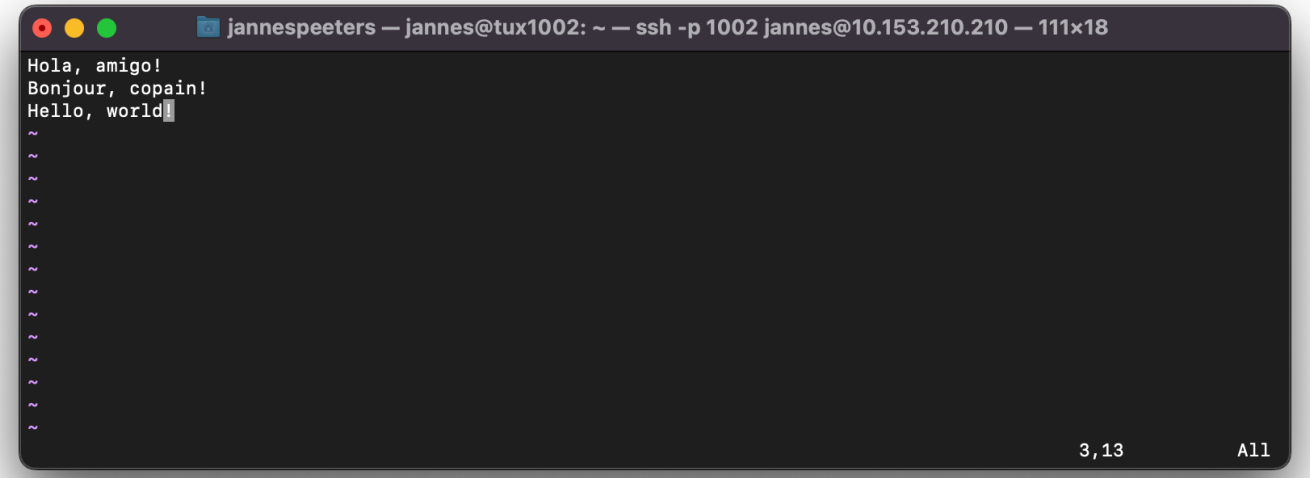
- **Bestand openen**:

  ```
  nano bestandsnaam.txt
  ```

- **Tekst opslaan**:
  Druk op `Ctrl + O` (druk op `Enter` om te bevestigen).
- **Tekst knippen**:
  Druk op `Ctrl + K` om de huidige regel te knippen.
- **Tekst plakken**:
  Druk op `Ctrl + U` om de eerder geknipte regel tekst te plakken.
- **Undo**
  Druk op `Alt + U` om de eerdere wijziging ongedaan te maken (`esc + U` on mac).

- **Redo**
  Druk op `Alt + E` om de eerder ongedaan gemaakte wijziging terug in te voeren (`esc + E` on mac).
- **Bestand sluiten**:
  Druk op `Ctrl + X` .

De `Vi` Editor



`vi` (en zijn verbeterde versie **`vim`**) is een krachtige teksteditor die standaard beschikbaar is in de meeste Linux- en Unix-systemen. Het belangrijkste kenmerk van `vi` is dat het een **modale editor** is, wat betekent dat het verschillende **modi** heeft voor navigeren, bewerken en commando's uitvoeren. Dit kan in het begin even wennen zijn, maar het biedt veel flexibiliteit.

Modi in `vi`

1. **Command Mode** (Opdrachtmodus):
   - Dit is de standaard modus wanneer je `vi` opent. In deze modus voer je commando's uit om te navigeren, tekst te bewerken of speciale acties uit te voeren.
   - **Navigatie** en **bewerkingsopdrachten** zoals het verwijderen van een regel of het kopiëren van tekst worden hier uitgevoerd.
2. **Insert Mode** (Invoermodus):
   - In deze modus kun je tekst typen en bewerken.
   - Om de invoermodus in te gaan, gebruik je een commando zoals `i` , `a` , of `o` .
   - Je keert terug naar de opdrachtmodus door op `Esc` te drukken.
3. **Visual Mode** (Selectiemodus):

    - Hiermee kun je tekst selecteren om deze te bewerken, te kopiëren of te verwijderen.
    - Activeren door `v` (voor selecties per teken) of `V` (voor selecties per regel) in de opdrachtmodus in te drukken.
4. **Command-Line Mode**:
    - In deze modus kun je commando's typen die beginnen met een `:` (dubbele punt), zoals `:w` om op te slaan, `:q` om te sluiten of `:wq` om op te slaan en te sluiten.
    - Deze modus bereik je door `:` in te drukken vanuit de opdrachtmodus.

Navigeren in `vi`

- **`h`, `j`, `k`, `l`** : Beweeg de cursor naar links, omlaag, omhoog, en naar rechts.
- **`0`** of **`^`** : Ga naar het begin van de regel.
- **`$`** : Ga naar het einde van de regel.
- **`w`** : Ga naar het begin van het volgende woord.
- **`b`** : Ga naar het begin van het vorige woord.
- **`G`** : Ga naar het einde van het bestand.
- **`gg`** : Ga naar het begin van het bestand.

Tekst bewerken

- **`i`** : Ga naar de invoermodus en voeg tekst in vóór de cursor.
- **`a`** : Ga naar de invoermodus en voeg tekst in **na** de cursor.
- **`o`** : Voeg een nieuwe regel onder de huidige regel toe en ga naar de invoermodus.
- **`x`** : Verwijder het teken onder de cursor.
- **`dd`** : Verwijder de huidige regel.
- **`yy`** : Kopieer (yank) de huidige regel.
- **`p`** : Plak de gekopieerde of uitgeknipte tekst **onder** de huidige regel.
- **`u`** : Ongedaan maken (undo) van de laatste bewerking.
- **`Ctrl + r`** : Herstellen (redo) van de ongedaan gemaakte bewerking.

Opslaan en Sluiten

- **`Esc`** : Ga terug naar de opdrachtmodus (indien je in de invoermodus zat).
- **`:`** gevolgd door een commando om een opdracht uit te voeren:
    - **`:w`** : Opslaan van het bestand (write).
    - **`:q`** : Sluiten van `vi` (quit).
    - **`:wq`** of **`:x`** : Opslaan en sluiten.
    - **`:q!`** : Sluiten zonder op te slaan (force quit).